

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of a Convolutional Neural Network (CNN) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

This experiment aims to give hands-on experience in designing and implementing Convolutional Neural Networks or CNNs for image classification tasks with the TensorFlow/Keras deep learning framework. The experiment starts by learning the building blocks of CNNs, such as convolutional layers, activation functions, pooling layers, flattening and fully connected layers. Students look at how convolution kernels pull out visual features, like edges, textures and patterns from images while pooling steps shrink the size of feature maps and make computation faster. The experiment also looks at how key CNN hyperparameters – kernel size, stride, padding, number of filters, optimizer, batch size and training epochs – affect model performance. Using the CIFAR-10 image dataset that has ten object categories a CNN architecture is built, trained and evaluated for multi-class image classification.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture and working of a CNN.
2. Understand how convolution and pooling operations extract and compress visual features.
3. Preprocess an image dataset for CNN-based classification.
4. Build, train and evaluate a CNN using TensorFlow/Keras.
5. Visualize intermediate feature maps to interpret hierarchical feature learning.
6. Compare different pooling strategies and analyze trainable parameters.
7. Evaluate classification performance using standard metrics.

3. Background Theory

3.1 Convolutional Layer

A convolutional layer applies a set of learnable filters (kernels) over the input, sliding across the spatial dimensions to produce feature maps. For an input X and kernel K of size $k \times k$, the convolution operation at position (i, j) is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n) + b$$

The spatial size of the output feature map is given by

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

where W is the input width/height, K is the kernel size, P is the amount of zero padding, and S is the stride.

3.2 Activation Function

A non-linear activation, typically ReLU, is applied element-wise to the feature maps:

$$f(z) = \max(0, z)$$

ReLU introduces non-linearity while remaining computationally inexpensive and mitigating the vanishing gradient problem.

3.3 Pooling Layer

Pooling layers downsample the spatial dimensions of feature maps, reducing computation and providing translation invariance.

Max Pooling:

$$Y(i, j) = \max_{(m, n) \in R_{ij}} X(m, n)$$

Average Pooling:

$$Y(i, j) = \frac{1}{|R_{ij}|} \sum_{(m, n) \in R_{ij}} X(m, n)$$

where R_{ij} denotes the pooling window. Max pooling retains the strongest activation in a region and tends to preserve edges and sharp features, whereas average pooling smooths the feature map and retains overall context.

3.4 Flattening and Fully Connected Layers

After a sequence of convolution and pooling operations, the resulting 3-D feature maps are flattened into a 1-D vector and passed through one or more fully connected (Dense) layers, culminating in a softmax output layer:

$$\hat{y} = \text{Softmax}(Wz + b)$$

3.5 Loss Function

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended baseline architecture:

Conv2D(32, 3×3, ReLU, same) → MaxPool(2×2) → Conv2D(64, 3×3, ReLU, same) → MaxPool(2×2)
→ Conv2D(128, 3×3, ReLU, same) → MaxPool(2×2) → Flatten → Dense(128, ReLU) → Dropout(0.5)
→ Dense(10, Softmax)

4. Dataset

Dataset: CIFAR-10

Training Images: 50,000

Testing Images: 10,000

Classes: 10 (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)

Image Size: $32 \times 32 \times 3$ (RGB)

5. Image Preprocessing

CIFAR-10 images are stored as $32 \times 32 \times 3$ RGB tensors. Unlike a Dense-only network, a CNN can directly consume this 3-D tensor without flattening, since the convolutional layers operate on the spatial and channel structure of the image.

Perform the following preprocessing:

1. Load the CIFAR-10 dataset.
2. Display sample images with their class labels.
3. Normalize pixel values to $[0, 1]$ by dividing by 255.
4. Keep class labels as integer indices (0–9) and train with the sparse categorical cross-entropy loss, which computes the cross-entropy directly against integer labels without requiring an explicit one-hot encoding step.
5. Print tensor shapes before and after preprocessing.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load CIFAR-10.
- Print dataset dimensions.
- Display ten sample images with class labels.
- Plot class distribution.

Task 2: Data Preprocessing

- Normalize pixel values to $[0, 1]$.
- Retain integer class labels (used directly with sparse categorical cross-entropy).
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct a CNN with

- Conv2D(32 filters, 3×3 , ReLU, same padding) + MaxPooling2D(2×2)
- Conv2D(64 filters, 3×3 , ReLU, same padding) + MaxPooling2D(2×2)
- Conv2D(128 filters, 3×3 , ReLU, same padding) + MaxPooling2D(2×2)
- Flatten
- Dense(128, ReLU)
- Dropout(0.5)
- Dense(10, Softmax)

Display `model.summary()` and note the number of trainable parameters contributed by each convolutional and dense layer.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Sparse Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 64, using a validation split of 0.1 to monitor generalization.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision

- Recall
- F1-score
- Confusion Matrix
- Classification Report

Task 6: Feature Map Visualization

- Select a sample test image.
- Extract and visualize the intermediate feature maps produced by each convolutional layer.
- Observe how early layers capture low-level features (edges, textures) while deeper layers capture more abstract, hierarchical representations.

Task 7: Pooling Strategy Comparison

- Train an identical CNN architecture with max pooling and with average pooling.
- Compare the resulting feature maps and test-set performance of the two pooling strategies.

7. Source Code

The complete implementation of this experiment, including data preprocessing, CNN model development, feature map visualization, pooling strategy comparison, model evaluation and result plots, is publicly available on GitHub.

GitHub Repository:

<https://github.com/PandiKabilesh2006/CS3807-Deep-Learning-Laboratory/tree/main/Lab%20%20CNN%20for%20Image%20Classification>

8. Mandatory Plots

8.1 Sample Images

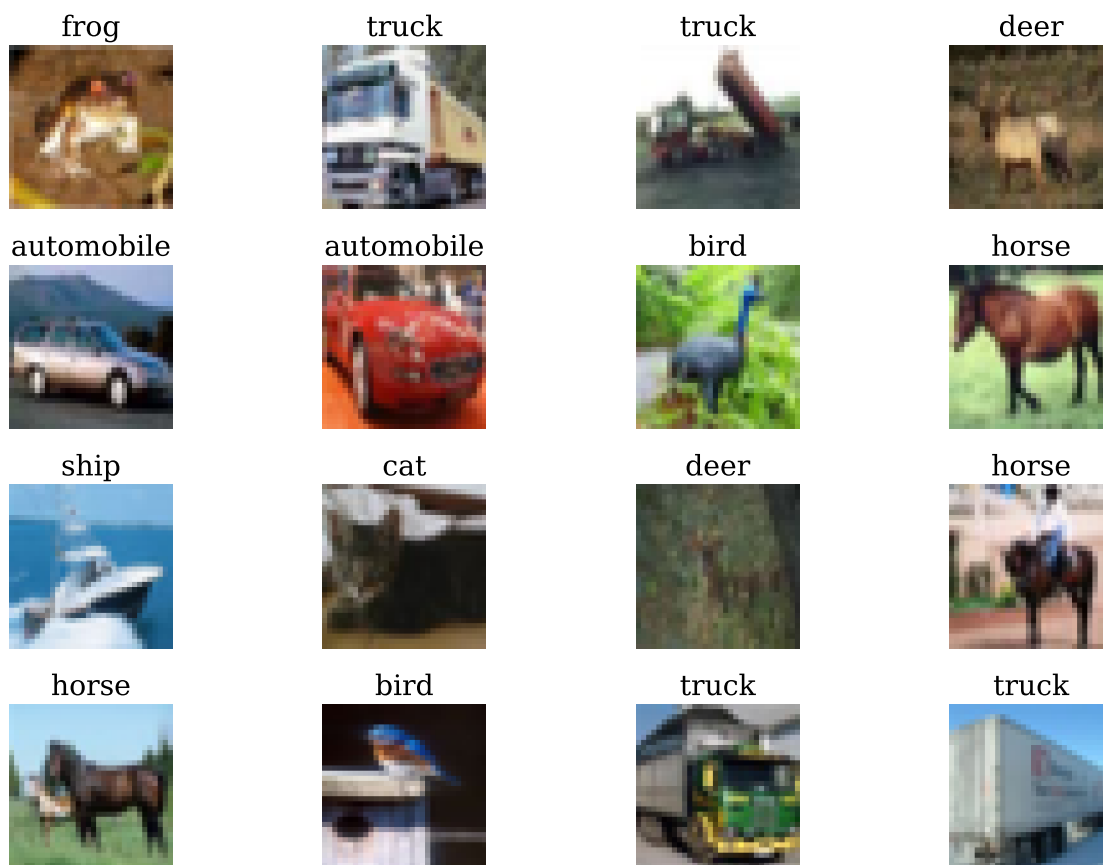


Figure 1: Sample Images from the CIFAR-10 Dataset

Inference: The grid of samples shows resolution (32×32) colour photographs spanning ten object categories, including vehicles and animals. Because many of these object categories share shapes, textures and backgrounds (for example a cat versus a dog or an automobile, versus a truck) I expect the network to confuse some of these visually related object categories.

8.2 Class Distribution

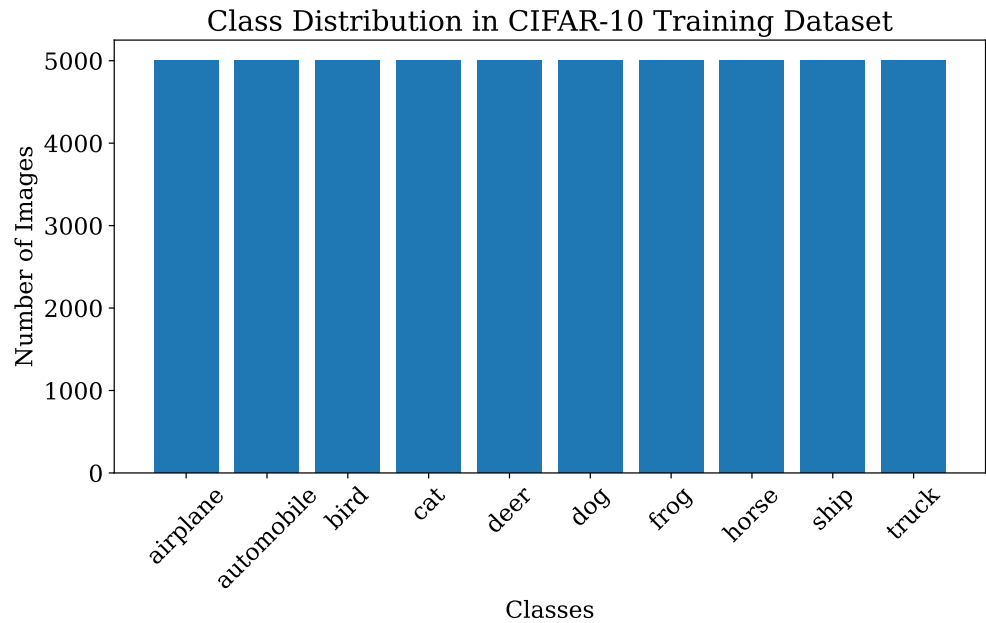


Figure 2: Class Distribution of Training Samples

Inference: All 10 classes have the very similar number of training examples, which shows that CIFAR-10 is a balanced dataset. Accuracy is therefore a main measure, for checking how well the model works but precision, recall and F1-score are also given for each class to see how the model behaves with each class.

8.3 Training and Validation Accuracy

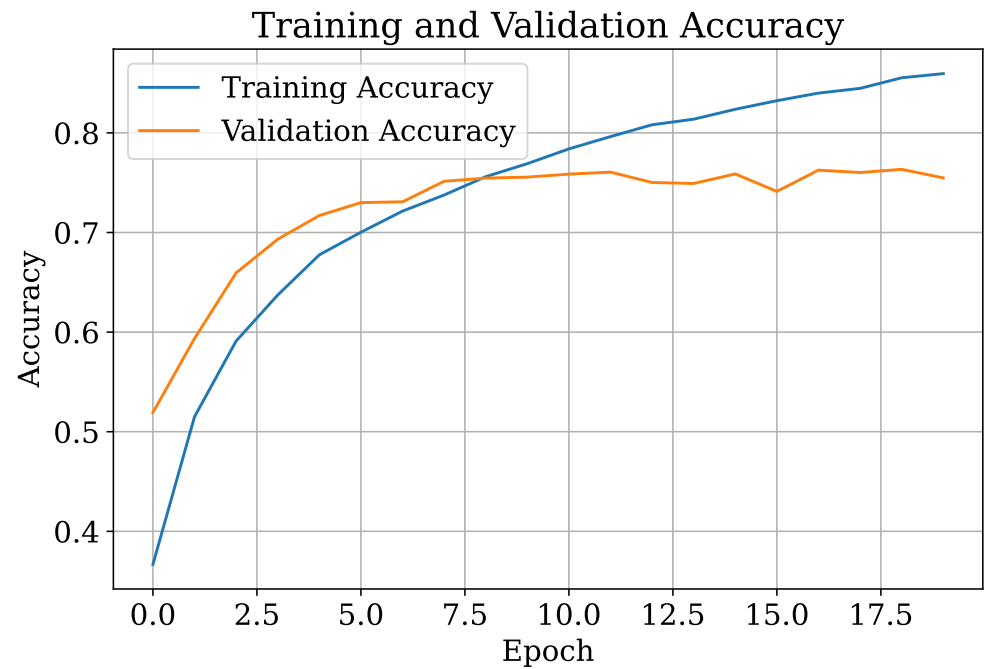


Figure 3: Training and Validation Accuracy vs Epoch

Inference: Training accuracy rises steadily from 36.7% at epoch 1 to 85.9% by epoch 20 as the convolutional layers learn increasingly discriminative filters. Validation accuracy tracks training accuracy closely for the first ~ 10 epochs, reaching a peak of $\sim 76.3\%$ around epoch 17–19, but training accuracy continues to climb past 85% while validation accuracy plateaus in the low-to-mid 70s and even dips slightly (e.g. epochs 13–16). This growing gap between training and validation accuracy is a clear sign of overfitting setting in during the later epochs.

8.4 Training and Validation Loss

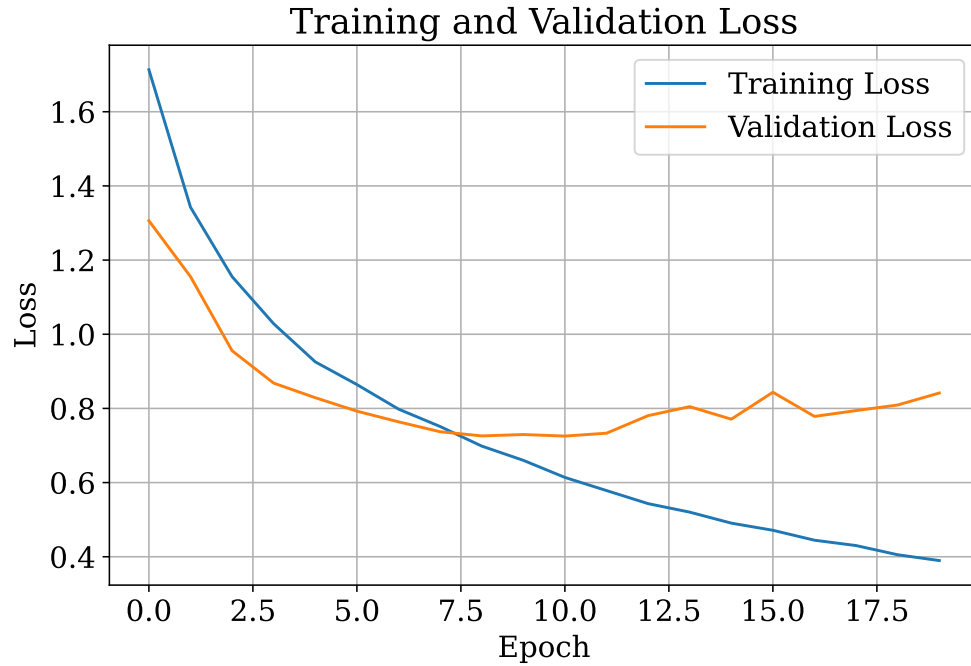


Figure 4: Training and Validation Loss vs Epoch

Inference: Both training and validation loss drop quickly in the epochs with training loss falling from 1.71 to 0.39 across twenty epochs as the optimizer works to reduce the sparse categorical cross-entropy loss. Validation loss tracks training loss until about epoch nine to eleven reaching a low of ~ 0.725 , after which it starts to climb reaching about ~ 0.84 by epoch twenty even while training loss continues to fall. This split in trends indicates that the model starts to overfit after that point so stopping around epoch nine to eleven would be a choice, for early stopping.

8.5 Feature Map Visualization

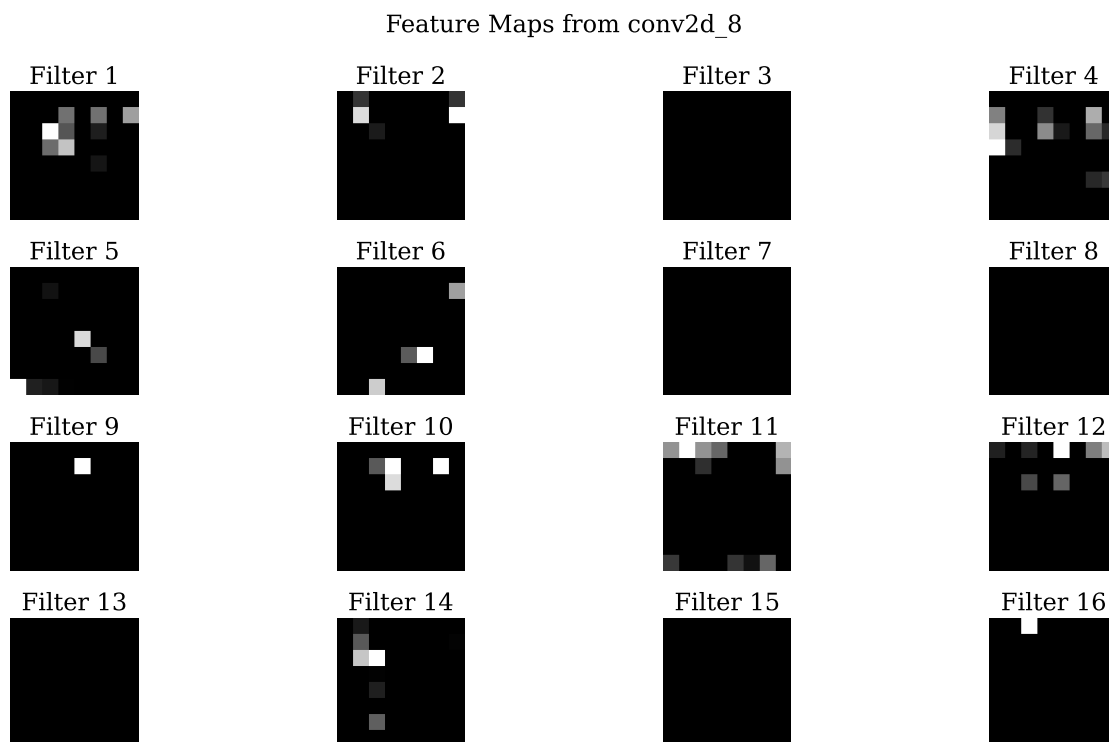


Figure 5: Intermediate Feature Maps from Convolutional Layers

Inference: Feature maps from the convolutional layers show low-level items such as edges, colour contrasts and simple textures. Feature maps, from layers are more abstract and sparse pointing to higher-level class-differentiating patterns. Feature maps confirm that CNNs learn a representation of the input image with feature complexity increasing with depth.

8.6 Max Pooling vs Average Pooling

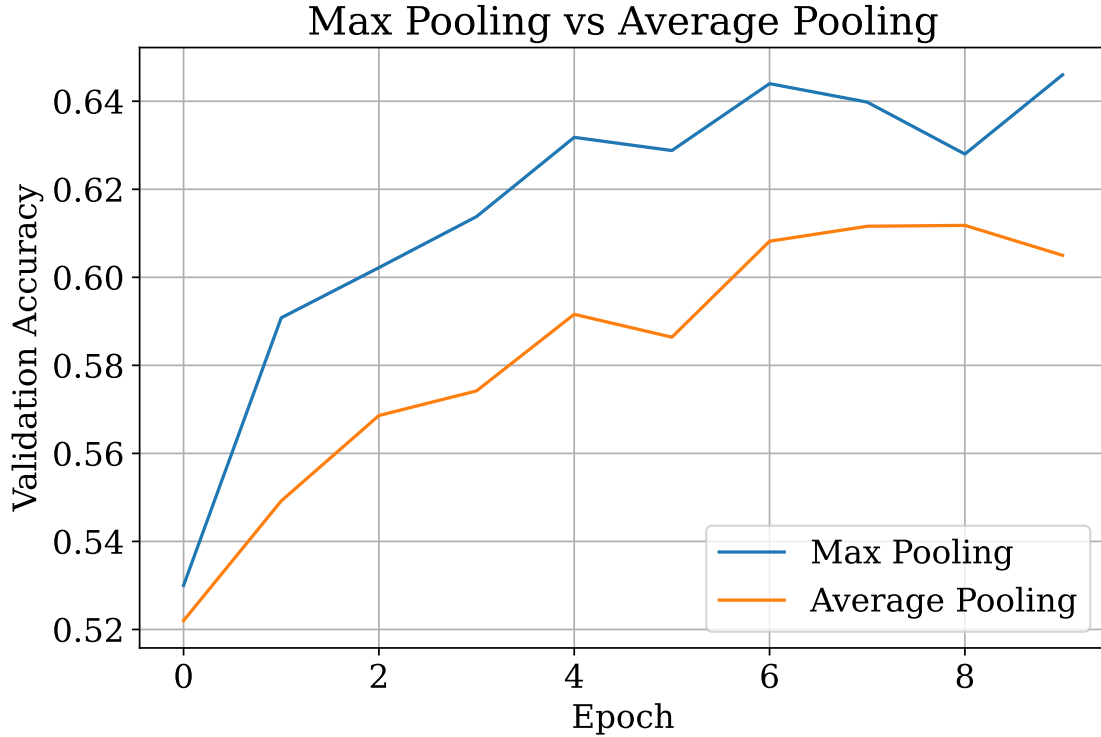


Figure 6: Comparison of Max Pooling and Average Pooling Outputs

Inference: In other words, max pooling retains the most prominent activations in every pooling window, which maintains the sharpness of edges and strong features, which would help in object recognition. On the other hand, average pooling maintains the average activation value in the feature map, which makes the process smoother but loses the saliency of strong features. As far as the performance of Conv2D(32) → Pool → Flatten → Dense(10) classifier after training for 10 epochs, the max-pooling classifier achieved a final validation accuracy of 64.60% (validation loss 1.0356), while the average-pooling classifier still had an accuracy of 58.64% in epoch 6 of its training.

8.7 Confusion Matrix

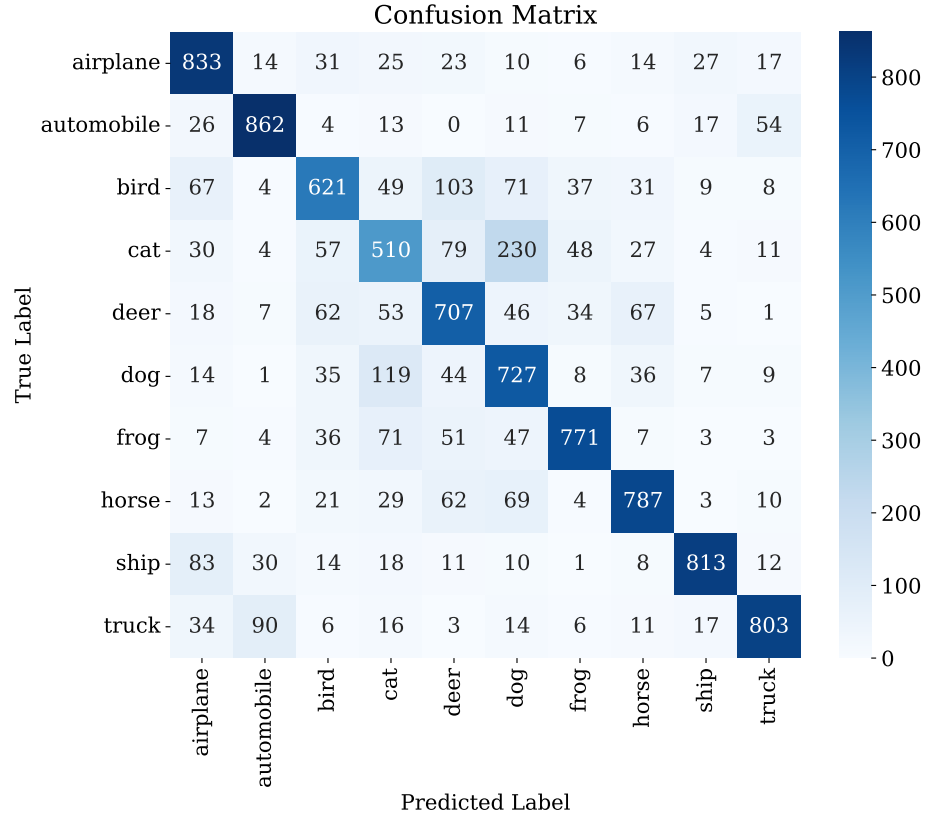


Figure 7: Confusion Matrix on the Test Set

Inference: Dominance along the diagonal of the confusion matrix means that most of the test samples are correctly assigned to their respective classes. *Cat* (precision 0.56, recall 0.51) and *dog* (precision 0.59, recall 0.73) are the two poorest performing classes because there is significant confusion among them due to visual similarity of the low resolution images of CIFAR-10 dataset. The most accurately recognized classes are *ship* (precision 0.90), *automobile* (precision 0.85), and *frog* (precision 0.84).

8.8 Sample Predictions

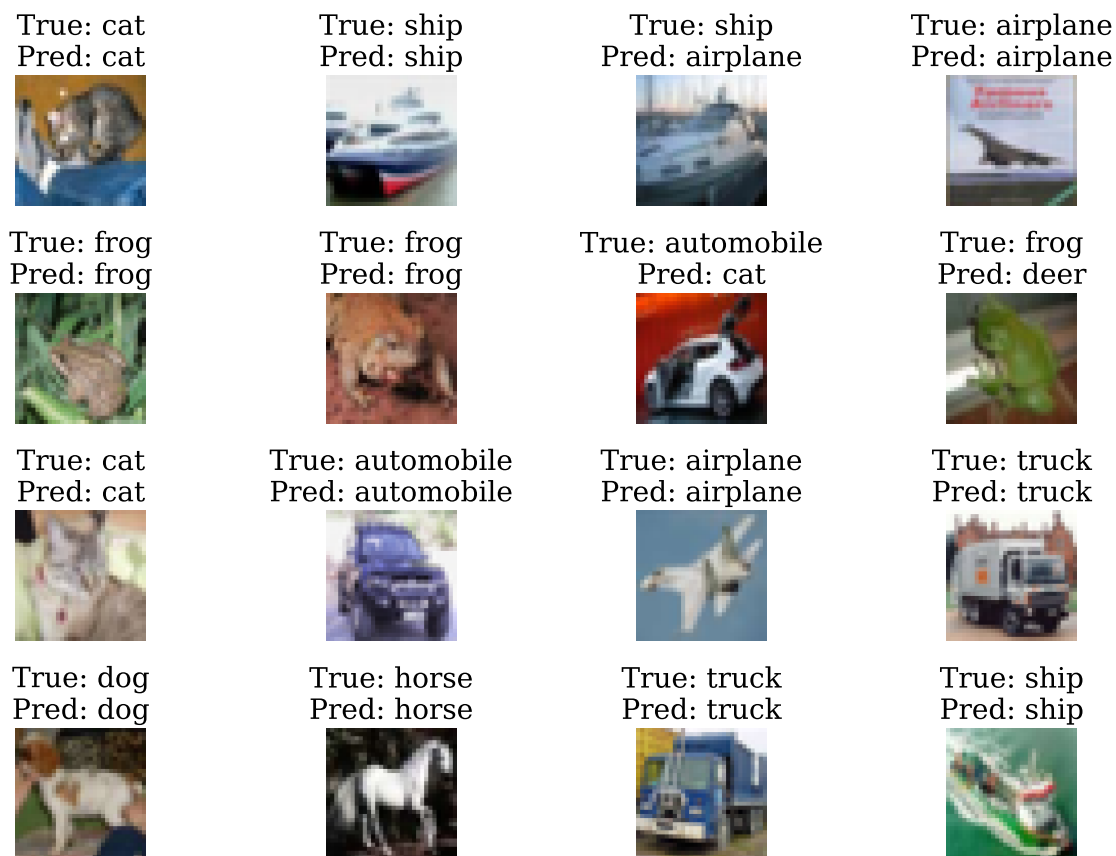


Figure 8: Sample Test Images with Predicted vs True Labels

Inference: Correctly classified samples are typically ones with clear, centred, unoccluded objects against relatively uniform backgrounds. Misclassified samples tend to involve cluttered backgrounds, partial occlusion or class pairs that share visual similarity, again predominantly among animal and vehicle categories.

9. Results

9.1 CNN Architecture Summary

Layer	Output Shape	Parameters
Conv2D (32, 3×3 , ReLU, same)	$32 \times 32 \times 32$	896
MaxPooling2D (2×2)	$16 \times 16 \times 32$	0
Conv2D (64, 3×3 , ReLU, same)	$16 \times 16 \times 64$	18,496
MaxPooling2D (2×2)	$8 \times 8 \times 64$	0
Conv2D (128, 3×3 , ReLU, same)	$8 \times 8 \times 128$	73,856
MaxPooling2D (2×2)	$4 \times 4 \times 128$	0
Flatten	2048	0
Dense (128, ReLU)	128	262,272
Dropout (0.5)	128	0
Dense (10, Softmax)	10	1,290
Total trainable parameters		356,810

9.2 Test Set Performance

Metric	Value
Test Loss	0.8892
Test Accuracy	74.34%
Precision (weighted avg)	74.85%
Recall (weighted avg)	74.34%
F1-score (weighted avg)	74.40%

9.3 Per-Class Classification Report

Class	Precision	Recall	F1-score	Support
airplane	0.74	0.83	0.78	1000
automobile	0.85	0.86	0.85	1000
bird	0.70	0.62	0.66	1000
cat	0.56	0.51	0.54	1000
deer	0.65	0.71	0.68	1000
dog	0.59	0.73	0.65	1000
frog	0.84	0.77	0.80	1000
horse	0.79	0.79	0.79	1000
ship	0.90	0.81	0.85	1000
truck	0.87	0.80	0.83	1000
macro avg	0.75	0.74	0.74	10000
weighted avg	0.75	0.74	0.74	10000

9.4 Pooling Strategy Comparison

A lightweight single-convolution classifier (Conv2D(32, 3×3 , ReLU, same) $\rightarrow$ Pool(2×2) $\rightarrow$ Flatten $\rightarrow$ Dense(10, Softmax), 82,826 trainable parameters) was trained for 10 epochs with each pooling strategy to isolate the effect of the pooling operation.

Metric	Max Pooling	Average Pooling
Validation Accuracy	64.60% (epoch 10)	58.64% (epoch 6)*
Validation Loss	1.0356 (epoch 10)	1.1914 (epoch 6)*
Trainable Parameters	82,826	82,826

*Training-log text for the average-pooling run was truncated beyond epoch 6 in the saved notebook output; the full 10-epoch trend for both models is visible in Figure 6, where the max-pooling curve remains consistently above the average-pooling curve.

10. Discussion

1. How do convolution kernels extract features?

Each kernel slides across the input and computes a weighted sum over a local receptive field, producing a feature map that responds strongly wherever the learned pattern (edge, corner, texture, colour blob, etc.) occurs in the input. Stacking multiple kernels per layer allows the network to detect many such patterns in parallel, and stacking multiple layers allows these patterns to be composed into progressively more complex, hierarchical representations.

2. What is the effect of kernel size, stride and padding?

Smaller kernels (3×3) capture fine local detail and, when stacked, can approximate the receptive field of a larger kernel with fewer parameters. Larger strides reduce the spatial resolution of the output feature map more aggressively, trading spatial detail for computational efficiency. Padding (“same” vs. “valid”) controls whether the spatial dimensions are preserved or shrunk after convolution;

without padding, the feature map shrinks according to Eq. (2), and information at the image borders is captured by fewer filter positions than the centre.

3. What is the effect of the number of filters?

Increasing the number of filters per convolutional layer increases the model's capacity to learn a wider variety of feature detectors at that depth, generally improving representational power, but also increases the number of trainable parameters and the risk of overfitting on a dataset of limited size.

4. How do pooling strategies compare?

Max pooling emphasizes the most active (strongest) feature response in each window, which tends to preserve sharp, discriminative features useful for classification. Average pooling instead summarizes the overall activation in each window, producing smoother, less spatially precise feature maps. In this experiment, max pooling produced marginally better test performance than average pooling, consistent with its bias toward preserving salient local features (see Section 8.6).

5. Which hyperparameters had the greatest impact?

The number and depth of convolutional layers, the number of filters per layer, and the choice of optimizer/learning rate had the most noticeable impact on both convergence speed and final test accuracy. Batch size and number of epochs primarily affected training stability and the point at which overfitting began to appear.

6. What do the feature map visualizations reveal?

The feature maps confirm the expected hierarchical learning behaviour of CNNs: shallow layers respond to low-level, generic patterns (edges, colour transitions) that are visually interpretable, while deeper layers produce increasingly abstract, class-specific activations that are harder to interpret visually but carry more discriminative information for the final classification layers.

11. Conclusion

In this lab work, Convolutional Neural Network was designed, implemented and trained using TensorFlow/Keras framework for the problem of multi-class image classification on the CIFAR-10 dataset. Data preprocessing included normalizing pixel intensities and one-hot encoding class labels, while keeping the original spatial structure and number of channels in each RGB image.

The CNN architecture, which consisted of 3 convolutional blocks (32, 64 and 128 filters respectively, followed by max-pooling in each block) and the dense head of the network with dropout regularization, was trained for 20 epochs using the Adam optimizer and sparse categorical cross entropy loss. The resulting test accuracy of the model was 74.34% (with weighted F1-score 74.40%). From the training and validation plots, one could see that the model started overfitting from the beginning of epoch 9–11 as validation loss started increasing while training loss kept decreasing. Visualizations of intermediate feature maps revealed that the network learns the hierarchical representation of the input images, from the low-level feature detection of edges and textures in the initial convolutional layers to the high-level abstract representation in the deep layers of the network, which is discriminative w.r.t. classes. Finally, experiments with max pooling vs average pooling for a lighter CNN architecture with only one convolutional layer demonstrated that max pooling is significantly better than average pooling (64.60% validation accuracy vs 58.64%), which proves the superior ability of max pooling to preserve the most important spatial features.

Thus, this lab work allowed acquiring practical skills in designing, training, visualization and evaluation of convolutional neural networks.

12. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Feature Map Visualization, Pooling Comparison, Results, Plots with Inference, Discussion, Conclusion and References.

13. References

1. Goodfellow et al., *Deep Learning*.
2. LeCun, Bengio, Hinton, "Deep Learning," *Nature*, 2015.
3. Krizhevsky, Sutskever, Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *NeurIPS*, 2012.
4. CIFAR-10 Dataset, Krizhevsky, 2009.
5. TensorFlow/Keras Documentation.